



Waipapa
Taumata Rau
**University
of Auckland**

Mathematical Building Blocks in Deep Learning

SOFTENG 753: Machine Learning Techniques
and Applications

>>> Semester 2, 2026

>>> Maryam Hemmati
m.hemmati@auckland.ac.nz

Content



- **Hello World Example**
- **Tensors**
- **Tensor Operations**
- **Gradient Based Optimization**
 - **Stochastic Gradient Descent**
 - **Backpropagation Algorithm**
- **Looking Back at Hello World Example**

Hello World Example

- Our Hello World example is a neural network that uses the machine learning library [Keras](#) to learn to [classify handwritten digits](#).
- Classify grayscale images of handwritten digits (28×28 pixels) into their 10 categories (0 through 9).
- We use **MNIST** dataset
 - It is a set of [60,000 training images](#), plus [10,000 test images](#)
 - It is a classic in the machine learning community, which has been around almost as long as the field itself and has been intensively studied.
 - Assembled by the National Institute of Standards and Technology (the NIST in MNIST) in the 1980s.

Hello World Example

- In machine learning
 - A *category* in a classification problem is called a [class](#).
 - *Data points* are called [samples](#).
 - The *class* associated with a specific sample is called a [label](#).



The **MNIST** dataset comes preloaded in [Keras](#), in the form of a set of four [NumPy arrays](#).

```
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

NumPy is a highly popular Python library for numerical computation. It is rarely used to implement modern machine learning algorithms, due to [lacking GPU](#) and [autodifferentiation](#) support, but [NumPy arrays](#) are often used as a numerical data exchange format.

Hello World Example

➤ Workflow:

- We will feed the neural network the training data, *train_images* and *train_labels*.
- The network will learn to associate images and labels.
- We will ask the network to produce predictions for *test_images*.
- We will verify whether these predictions match the labels from *test_labels*.

Let's try the example in [Jupyter notebook](#)

Hello World Example

- The core building block of neural networks is the *layer*.
- Layers extract *meaningful representations* out of the data fed into them.
- Most of deep learning consists of chaining together simple layers that will implement a form of *progressive data distillation*.
- Dense layers are *fully connected* neural layers.
- The second layer is a *10-way softmax classification layer*
 - It will return an array of 10 *probability scores* (summing to 1).
 - Each score will be the probability that the current digit image belongs to one of our 10 digit classes.

```
import keras
from keras import layers

model = keras.Sequential(
    [
        layers.Dense(512, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ]
)
```

Our model consists of a sequence of *two Dense layers*

Hello World Example

- To make the model ready for training, we need to pick three more things, as part of the **compilation** step
- **A loss function**
 - How the model will be able to measure its performance on the training data and thus how it will be able to steer itself in the right direction.
- **An optimizer**
 - The mechanism through which the model will update itself based on the training data it sees, to improve its performance.
- **Metrics to monitor during training and testing**
 - Here, we will only care about **accuracy**
 - The fraction of the images that were correctly classified.

```
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"],  
)
```

Hello World Example

- Preprocess the data by
 - Reshaping it into the shape the model expects
 - Scaling it so that all values are in the **[0, 1]** interval.
- Our training images were stored in an array of shape **(60000, 28, 28)**
 - We transform it into an array of shape **(60000, 28 * 28)**
- Our training images were of type **uint8** with values in the **[0, 255]** interval.
 - We transform it into a **float32** array with values between 0 and 1.

```
train_images = train_images.reshape((60000, 28 * 28))  
train_images = train_images.astype("float32") / 255  
test_images = test_images.reshape((10000, 28 * 28))  
test_images = test_images.astype("float32") / 255
```

Before training, we will **preprocess** the data

```
model.fit(train_images, train_labels, epochs=5, batch_size=128)
```

Training a model in **Keras** is done via a call to the model's **fit()** method
We fit the model to its training data.

Hello World Example

- Two quantities are displayed during training:
 - The **loss** of the model over the training data
 - The **accuracy** of the model over the training data.
- The example quickly reaches an accuracy of 0.989 (98.9%) on the **training data**.
- We can use the trained model to predict class probabilities for new digits from the **test set**.

```
>>> test_digits = test_images[0:10]
>>> predictions = model.predict(test_digits)
>>> predictions[0]
array([[1.0726176e-10, 1.6918376e-10, 6.1314843e-08, 8.4106023e-06,
        2.9967067e-11, 3.0331331e-09, 8.3651971e-14, 9.9999106e-01,
        2.6657624e-08, 3.8127661e-07], dtype=float32)
```

Each number of index **i** in that array corresponds to the probability that digit image **test_digits[0]** belong to class **i**.

This first test digit has the highest probability score (0.99999106, almost 1) at **index 7**, so according to our model, it must be a **7**.

Hello World Example

- We can check the model performance by computing **average accuracy** over the entire **test set**.
- The test set accuracy is **97.8%**
- It is almost **double the error rate** of the training set (at 98.9% accuracy).
- This gap between training accuracy and test accuracy is an example of **overfitting**
- **Overfitting**: Machine learning models tend to perform worse on new data than on their training data.

```
>>> test_loss, test_acc = model.evaluate(test_images, test_labels)
>>> print(f"test_acc: {test_acc}")
test_acc: 0.9785
```

The test set accuracy is **97.8%**

Data Representations –Tensors

- All current machine learning systems use **tensors** as their basic data structure.
- Multidimensional NumPy arrays are called **tensors**.
- A tensor is a container for data, usually numerical data.
- Matrices are **rank-2 tensors**.
- Tensors are a generalization of matrices to an **arbitrary number of dimensions**.
 - In the context of tensors, a dimension is often called an **axis**.

The **number of axes** of a tensor is also called its **rank**.

Tensors

- **Scalars (rank-0 tensors)**
 - A tensor that contains only one number is called a **scalar** (scalar tensor, rank-0 tensor, or 0D tensor)
 - In NumPy, a float32 or float64 number is a scalar tensor.
 - A scalar tensor has **0 axes**.
- **Vectors (rank-1 tensors)**
 - An array of numbers is called a **vector** (rank-1 tensor or 1D tensor).
 - A **rank-1 tensor** has exactly **one axis**.
- **Matrices (rank-2 tensors)**
 - An array of vectors is a **matrix** (rank-2 tensor or 2D tensor).
 - A matrix has **two axes** (often referred to as **rows** and **columns**).
 - Matrix can be visually interpreted as a **rectangular grid of numbers**.
- **Rank-3 tensors and higher-rank tensors**
 - If we pack matrices in a new array, we obtain a **rank-3 tensor** (3D tensor)
 - They can be visually interpreted as a **cube of numbers**.

Tensors - Key attributes

➤ A tensor is defined by **three key attributes**:

➤ **Number of axes (rank)**

- For instance, a **rank-3 tensor** has **three axes**, and a matrix has two axes.
- This is called the tensor's **ndim** in Python libraries such as NumPy, JAX, TensorFlow, and PyTorch.

➤ **Shape**

- This is a tuple of integers that describes **how many dimensions** the tensor has **along each axis**.
- The previous matrix example has shape (3, 5), and the rank-3 tensor example has shape (3, 3, 5).
- A vector has a shape with a single element, such as (5,).
- A scalar has an empty shape, ().

➤ **Data type**

- This is the type of the data contained in the tensor.
- It is usually called **dtype** in Python libraries.
- A tensor's type could be float16, float32, float64, uint8, bool, and so on.
- In TensorFlow, you are also likely to come across string tensors.

Tensors - Manipulating in NumPy

➤ Selecting specific elements in a tensor is called **tensor slicing**.

➤ This example selects digits **#10 to #100** and puts them in an array of shape **(90, 28, 28)**
➤ **#100 is not included**

➤ In more detailed notation, we can specify a start index and stop index for the slice **along each tensor axis**.

➤ Note that **:** is equivalent to **selecting the entire axis**.

```
>>> my_slice = train_images[10:100]
>>> my_slice.shape
(90, 28, 28)
```

```
>>> # Equivalent to the previous example
>>> my_slice = train_images[10:100, :, :]
>>> my_slice.shape
(90, 28, 28)

>>> # Also equivalent to the previous example
>>> my_slice = train_images[10:100, 0:28, 0:28]
>>> my_slice.shape
(90, 28, 28)
```

More **detailed notation** equivalent to the top example

Tensors - Manipulating in NumPy

- We may select slices between **any two indices** along **each tensor axis**.

```
my_slice = train_images[:, 14:, 14:]
```

Selecting 14×14 pixels in the **bottom-right** corner of all images

- It is also possible to use **negative** indices.
 - Like negative indices in Python lists, they indicate **a position relative to the end** of the current axis.

```
my_slice = train_images[:, 7:-7, 7:-7]
```

Cropping the images to patches of 14×14 pixels **centered in the middle**

Tensors - Notion of Data Batches

- In general, **the first axis** (axis 0) in all data tensors in deep learning is the **samples axis**.
- In the MNIST example, **samples** are **images of digits**.
- Deep learning models do not process an entire dataset at once.
 - They break the data into small **batches** or groups of samples with a fixed size.
- When considering such a batch tensor, the first axis (axis 0) is called the **batch axis** (or batch dimension).

```
batch = train_images[:128]
```

One batch of our **MNIST** digits, with a batch size of **128**

```
batch = train_images[128:256]
```

The next batch

```
n = 3
batch = train_images[128 * n : 128 * (n + 1)]
```

The **n**th batch

Tensors - Real-World Examples

➤ Vector data

- Rank-2 tensors of shape (samples, features)
- Each sample is a vector of numerical attributes ("features")

➤ Timeseries data or sequence data

- Rank-3 tensors of shape (samples, timesteps, features)
- Each sample is a sequence (of length timesteps) of feature vectors

➤ Images

- Rank-4 tensors of shape (samples, height, width, channels)
- Each sample is a 2D grid of pixels, and each pixel is represented by a vector of values ("channels")

➤ Video

- Rank-5 tensors of shape (samples, frames, height, width, channels)
- Each sample is a sequence (of length frames) of images

Tensors – Vector Data

➤ Vector data is one of the most common cases.

- Each single data point can be encoded as a vector.
- A batch of data will be encoded as a rank-2 tensor.

➤ An actuarial dataset of people

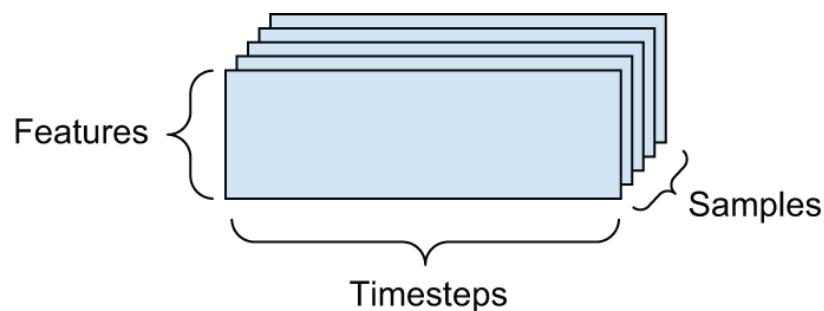
- Considers each person's age, gender, and income.
- Each person can be characterized as a vector of three values.
- The entire dataset of 100,000 people can be stored in a rank-2 tensor of shape (100000, 3).

➤ A dataset of text documents

- Represents each document by the counts of how many times each word appears in it (out of a dictionary of 20,000 common words).
- Each document can be encoded as a vector of 20,000 values (one count per word in the dictionary)
- The entire dataset of 500 documents can be stored in a tensor of shape (500, 20000).

Tensors – Timeseries Data

- When **time** (or the notion of **sequence order**) matters in the data, better to store it in a rank-3 tensor with an **explicit time axis**.
 - Each sample can be encoded as a **sequence** of vectors (a **rank-2 tensor**).
 - A batch of data will be encoded as a **rank-3 tensor**.
 - The time axis is **always** the **second axis** (axis of index 1), by convention.



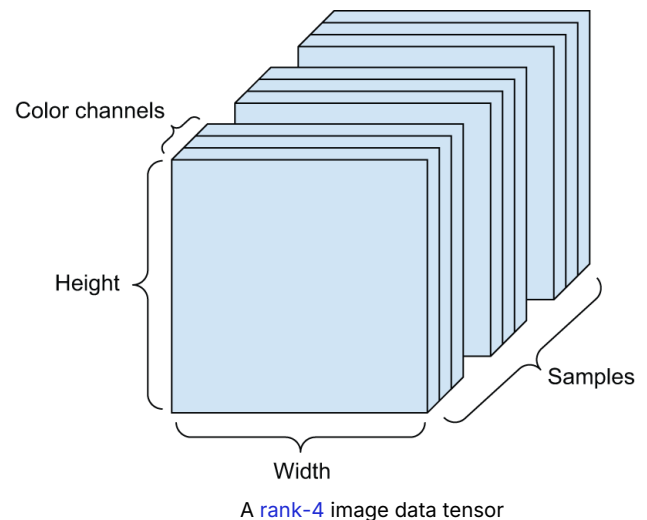
A **rank-3** timeseries data tensor

Tensors – Timeseries Data

- A dataset of **stock prices**
 - Every **minute**, we store the **current price of the stock**, the **highest price** in the past minute, and the **lowest price** in the past minute.
 - Every minute is encoded as a 3D vector
 - An entire day of trading is encoded as a matrix of shape **(390, 3)** (390 minutes in a trading day)
 - And 250 days' worth of data can be stored in a rank-3 tensor of shape **(250, 390, 3)**.
 - Each sample would be one day's worth of data.
- A dataset of **tweets**,
 - Each tweet is encoded as a **sequence of 280 characters** out of an alphabet of **128 unique characters**
 - Each character can be encoded as a **binary vector of size 128** (an all-zeros vector except for a 1 entry at the index corresponding to the character).
 - Then each tweet can be encoded as a rank-2 tensor of shape **(280, 128)**.
 - A dataset of 1 million tweets can be stored in a tensor of shape **(1000000, 280, 128)**.

Tensors – Image Data

- Images typically have **three dimensions**
 - **Height**
 - **Width**
 - **Color Depth**
- Grayscale images (like MNIST digits) have only a single color channel and could thus be stored in rank-2 tensors.
- By convention image tensors are **always rank-3**
 - One-dimensional color channel for grayscale images.
 - A batch of 128 **grayscale images** of size 256×256 could thus be stored in a tensor of shape **(128, 256, 256, 1)**
 - A batch of 128 **color images** could be stored in a tensor of shape **(128, 256, 256, 3)**



Tensors – Image Data

- There are **two conventions** for the shapes of image tensors
 - **The channels-last convention**
 - It is standard in **JAX** and **TensorFlow**, as well as most other deep learning tools.
 - **The channels-first convention**
 - It is standard in **PyTorch**.
- The **channels-last convention** places the color-depth axis at the end
 - **(samples, height, width, color_depth)**.
- The **channels-first convention** places the color depth axis right after the batch axis
 - **(samples, color_depth, height, width)**.
 - With the channels-first convention, the previous examples would become **(128, 1, 256, 256)** and **(128, 3, 256, 256)**.
 - The **Keras** API **provides support for both formats**.

Tensors – Video Data

- Video data is one of the few types of real-world data for which we need **rank-5 tensors**.
- A video can be understood as a **sequence of frames**, each **frame being a color image**.
 - Each frame can be stored in a **rank-3 tensor** (**height, width, color_depth**).
 - A sequence of frames can be stored in a **rank-4 tensor** (**frames, height, width, color_depth**).
 - A batch of different videos can be stored in a **rank-5 tensor** of shape (**samples, frames, height, width, color_depth**).
- A 60-second, 144×256 YouTube video clip sampled at 4 frames per second would have 240 frames.
 - A batch of four such video clips would be stored in a tensor of shape (**4, 240, 144, 256, 3**).
 - This is a total of 106,168,320 values!
 - If the **dtype** of the tensor was **float32**, then each value would be stored in 32 bits
 - The tensor would represent 425 MB.
- Videos in real life are much lighter because they are **not stored in float32** and they are typically **compressed** by a large factor (such as the MPEG format).

Tensor Operations

- All transformations learned by deep neural networks can be reduced to a handful of **tensor operations**.
- In the initial example, we built our model by **stacking Dense layers** on top of each other.
- We use three tensor operations here:
 - **A tensor product** (**matmul**) between the input tensor and a tensor named **W**.
 - **An addition** (+) between the resulting matrix and a vector **b**.
 - **A relu operation**: $\text{relu}(x)$ is **$\max(x, 0)$** .
 - **relu** stands for **REctified Linear Unit**.

```
keras.layers.Dense(512, activation="relu")
```

Each layer can be interpreted as a **function**, which takes as input a matrix and returns another matrix

```
output = relu(matmul(input, W) + b)
```

W is a **matrix** and **b** is a **vector**, both attributes of the layer

Tensor Operations - Element-Wise Operations

- The *relu* operation and *addition* are element-wise operations
- Operations that are applied *independently to each entry* in the tensors being considered.
- These operations are highly amenable to massively *parallel implementations*.
 - Vectorized implementations
- We can use a *for* loop to write a naive Python implementation of an element-wise operation.
- These operations are available as *well-optimized built-in NumPy* functions.

```
import numpy as np

# Element-wise addition
z = x + y
# Element-wise relu
z = np.maximum(z, 0.0)
```

We can do element-wise multiplication, subtraction, and so on

Tensor Operations - Broadcasting

- When the shapes of the two tensors being added differ the smaller tensor will be *broadcast* to match the shape of the larger tensor
 - In the Dense layer introduced earlier, we added a *rank-2 tensor* with a *vector*.
- **Broadcasting** consists of two steps:
 - Axes (called broadcast axes) are added to the smaller tensor to match the *ndim* of the larger tensor.
 - The smaller tensor is *repeated* alongside these new axes to match the full shape of the larger tensor.

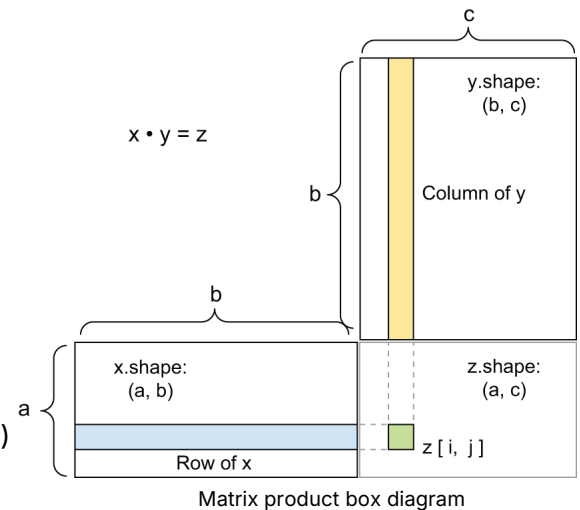
```
import numpy as np

# x is a random tensor with shape (64, 3, 32, 10).
x = np.random.random((64, 3, 32, 10))
# y is a random tensor with shape (32, 10).
y = np.random.random((32, 10))
# The output z has shape (64, 3, 32, 10) like x.
z = np.maximum(x, y)
```

The element-wise maximum operation to two tensors of **different** shapes via *broadcasting*.

Tensor Operations – Tensor Product

- The **tensor product** is one of the most common and most useful tensor operations.
- Also called **dot product** or **matmul** (short for *matrix multiplication*).
- In mathematical notation, we note the operation with a **dot (•)**
 - Hence the name **dot product**.
- In **NumPy**, a tensor product is done using the **np.matmul** function
- In **Keras**, it is done with the **keras.ops.matmul** function.
- We can take the product of two matrices **x** and **y** (**matmul(x, y)**) if and only if
 - **x.shape[1] == y.shape[0]**.
 - The result is a matrix with shape **(x.shape[0], y.shape[1])**
 - The coefficients are the vector products between the rows of **x** and the columns of **y**.



The rows of **x** and the columns of **y** must have the same size.

Tensor Operations – Tensor Reshaping

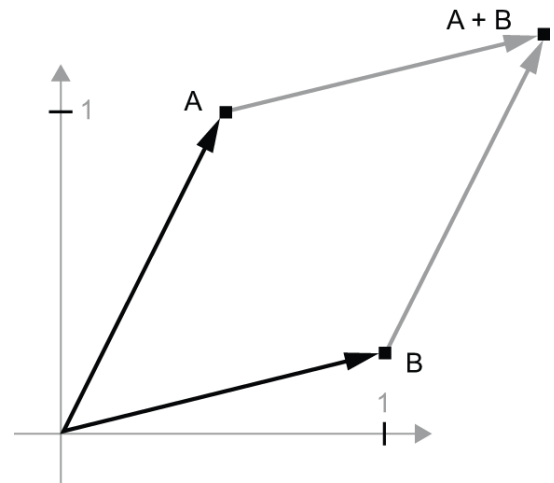
- **Tensor reshaping** is the other type of tensor operation that is essential in Deep Learning.
- Reshaping a tensor means **rearranging its rows and columns** to **match** a target shape.
- The reshaped tensor has the **same total number of coefficients** as the initial tensor.
- We used it when we preprocessed the digits data before feeding it into our model.
- A special case of reshaping commonly used is **transposition**.
- **Transposing** a matrix means **exchanging its rows and its columns**, so that **x[i, :]** becomes **x[:, i]**.

```
>>> # Creates an all-zeros matrix of shape (300, 20)
>>> x = np.zeros((300, 20))
>>> x = np.transpose(x)
>>> x.shape
(20, 300)
```

Transposing a matrix

Tensor Operations - Geometric Interpretation

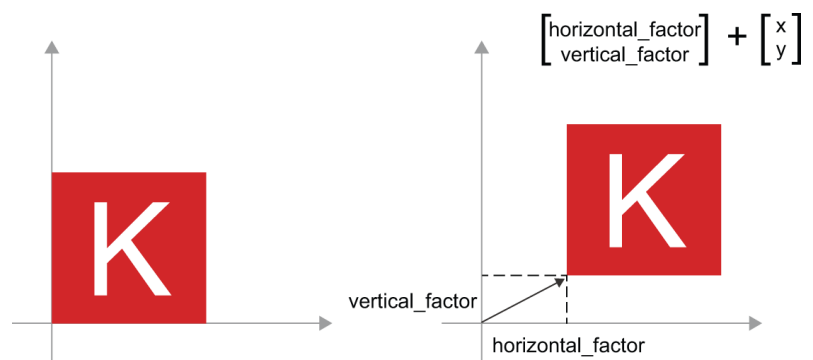
- The contents of the tensors manipulated by tensor operations can be interpreted as coordinates of points in some geometric space.
 - All tensor operations have a geometric interpretation.
- In general, elementary geometric operations can be expressed as tensor operations.
 - **Translation**
 - **Rotation**
 - **Scaling**
 - **Skewing**
 - and so on...



Geometric interpretation of the sum of two vectors.

Tensor Operations - Translation

- Adding a vector to a point will move the point by a fixed amount in a fixed direction.
- Adding a vector to a set of points (such as a 2D object), is called a **Translation**.

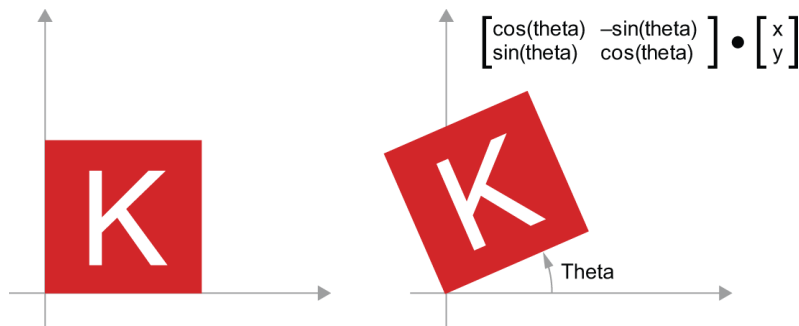


2D translation as a vector addition.

Tensor Operations - Rotation

- A counterclockwise **rotation** of a 2D vector by an angle θ is achieved via a **product with a 2×2 matrix**.

- **Rotation** matrix $R =$
$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$



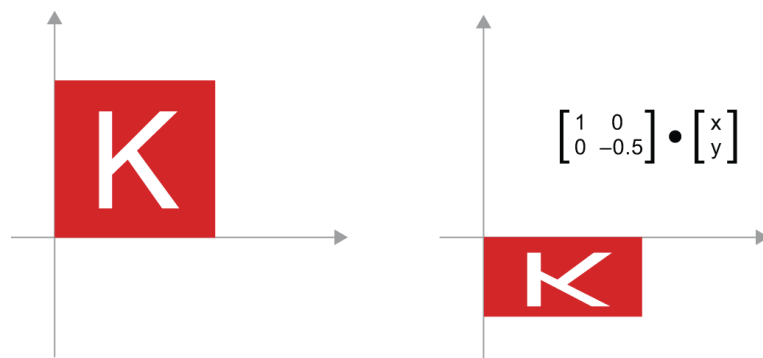
2D **rotation** (counterclockwise) as a **matrix product**.

Tensor Operations - Scaling

- A **vertical and horizontal scaling** of the image is achieved via a **product with a 2×2 matrix**.

- **Scaling** matrix $S =$
$$\begin{bmatrix} \text{horizontal_factor} & 0 \\ 0 & \text{vertical_factor} \end{bmatrix}$$

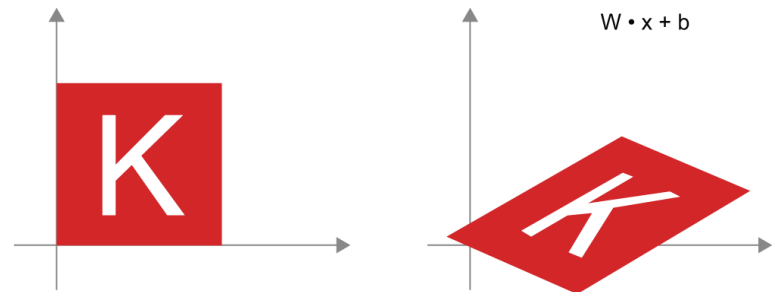
- Such a matrix is called a **diagonal matrix** because it only has non-zero coefficients in its **diagonal**, going from the top left to the bottom right.



2D **scaling** as a **matrix product**.

Tensor Operations - Affine

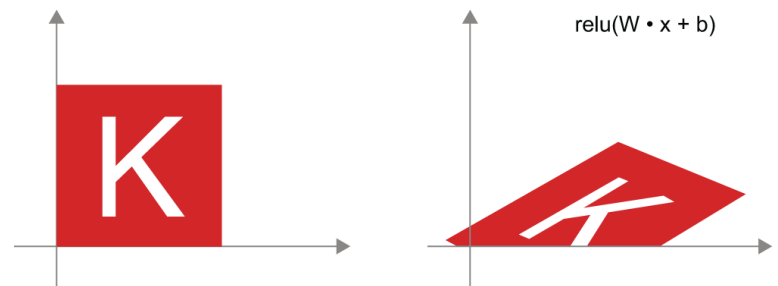
- A product with an arbitrary matrix implements a **linear transform**.
 - **Scaling** and **rotation** are by definition, linear transforms.
- An **affine transform** is the combination of a **linear transform** (achieved via a matrix product) and a **translation** (achieved via a vector addition).
- That is exactly the **$y = W @ x + b$** computation implemented by the Dense layer.
 - A **Dense layer without an activation function** is an affine layer.



Affine transform in the plane.

Tensor Operations – Affine & Activation

- If we apply **many affine transforms** repeatedly, we still end up with **an affine transform**.
- So, we could just have applied that **one affine transform** in the first place!
- **$\text{affine2}(\text{affine1}(x)) = W2 @ (W1 @ x + b1) + b2 = (W2 @ W1) @ x + (W2 @ b1 + b2)$** .
- A multilayer neural network made entirely of **Dense layers without activations** would be **equivalent** to a **single Dense layer**.
 - This is why we need activation functions, like **relu**.



Affine transform followed by **relu** activation

Gradient Based Optimization

- The **weights** (trainable parameters) contain the information **learned** by the model from exposure to training data.
 - Initially, these weight matrices are filled with small **random** values.
 - We gradually adjust these weights, based on a feedback signal.

➤ Training Loop

- Draw a batch of training samples **x** and corresponding targets **y_true**.
- Run the model on **x** (a step called the **forward pass**) to obtain predictions **y_pred**.
- Compute the **loss** of the model on the batch, a measure of the mismatch between **y_pred** and **y_true**.
- Update all weights of the model in a way that **slightly reduces the loss** on this batch.

```
output = relu(matmul(input, W) + b)
```

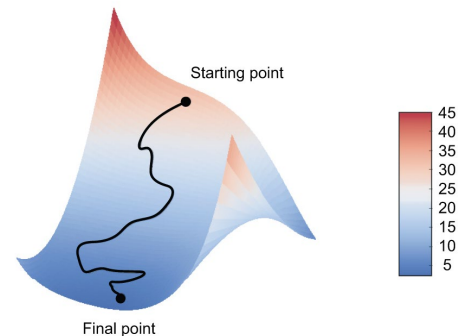
W and **b** are tensors that are attributes of the layer. They are called the **kernel** and **bias** attributes, respectively).

Gradient Based Optimization

- **Gradient descent** is the optimization technique that powers modern neural networks.
- All of the functions used in our models (such as **matmul** or **+**) transform their input in a **smooth** and **continuous** way.
- These functions and a chain of these function are **differentiable**.
- We use a mathematical operator called the **gradient** to describe how the loss varies as you move the model's coefficients in different directions.
- If we compute this gradient, we can use it to **move the coefficients in a direction that decreases the loss**.
- We can reduce **loss_value = f(W)** by moving **W** in the opposite direction from the gradient
 - Such as an update of **W1 = W0 - step * grad(f(W0), W0)** where step is a **small scaling factor**.
- The scaling factor step is needed because **grad(loss_value, W0)** only **approximates** the curvature when we are close to **W0**.

Gradient Based Optimization

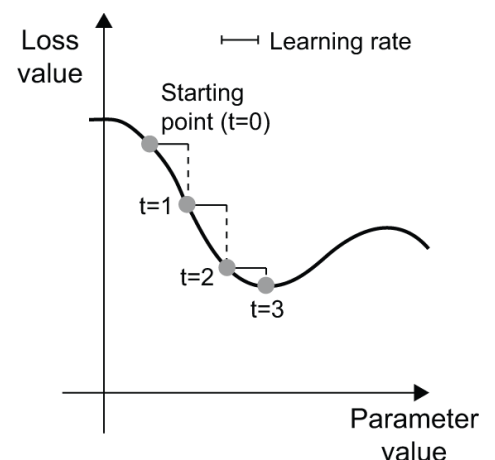
- Given a differentiable function, it's theoretically possible to find its minimum **analytically**.
 - A function's **minimum** is a point where the **derivative is 0**.
- Finding **analytically** the **combination of weight values** that yields the smallest possible loss function is intractable for real neural networks.
 - This is a polynomial **equation of N variables**, where N is the number of coefficients in the model.
- Instead, we can use the **four-step algorithm** outlined earlier.



Gradient descent down a 2D loss surface (two learnable parameters)

Gradient Based Optimization

- In **mini-batch stochastic gradient** descent, step 4 in our algorithm is replaced by:
 - **Compute the gradient of the loss** with regard to the model's parameters (**backward pass**).
 - **Move the parameters** a little in the **opposite direction from the gradient**
$$W -= \text{learning_rate} * \text{gradient}$$
- The learning rate (learning_rate here) is a scalar factor modulating the **speed** of the **gradient descent** process.
 - If too small, the descent down the curve will take many iterations, and it could get stuck in a local minimum.
 - If too large, our updates may end up taking us to completely random locations on the curve.



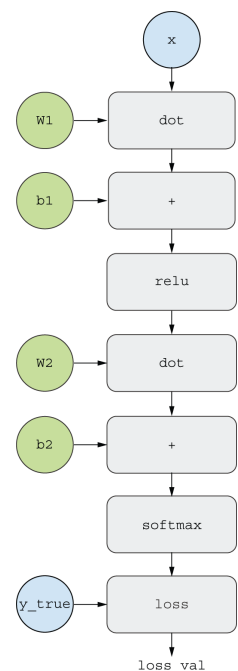
SGD down a 1D loss curve (one learnable parameter)

Gradient Based Optimization

- **Backpropagation** is a way to use the derivative of simple operations (such as addition, relu, or tensor product) to easily compute the gradient of **arbitrarily complex combinations** of these atomic operations.
- The **Chain Rule**:
 - Consider two functions **f** and **g**, as well as the composed function **fg** such that **$y = fg(x) == f(g(x))$**
 - Chain rule states that **$grad(y, x) == grad(y, x1) * grad(x1, x)$** .
 - This enables us to compute the derivative of **fg** as long as we **know the derivatives** of **f** and **g**.

Gradient Based Optimization

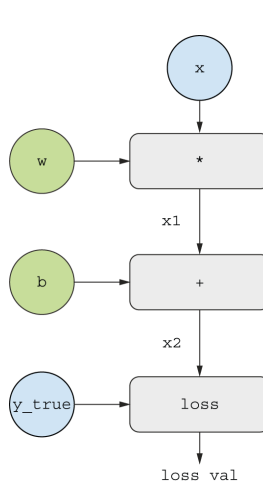
- A **computation graph** is the data structure at the heart of the deep learning revolution.
- It is a **directed acyclic graph** of operations
 - In our case, **tensor operations**.



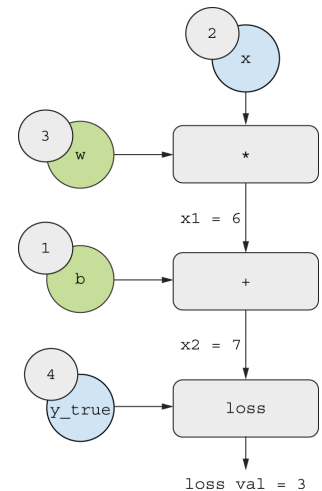
The **computation graph** representation of our two-layer model

Gradient Based Optimization

- We consider a simplified version of the graph, where we only have one linear layer and where all variables are scalar.
- We will take
 - Two scalar variables w , b ,
 - A scalar input x ,
 - and apply some operations to them to combine into an output y .
- We will apply an absolute value error loss function:
 - $\text{loss_val} = \text{abs}(y_{\text{true}} - y)$.
- We are interested in computing $\text{grad}(\text{loss_val}, b)$ and $\text{grad}(\text{loss_val}, w)$.



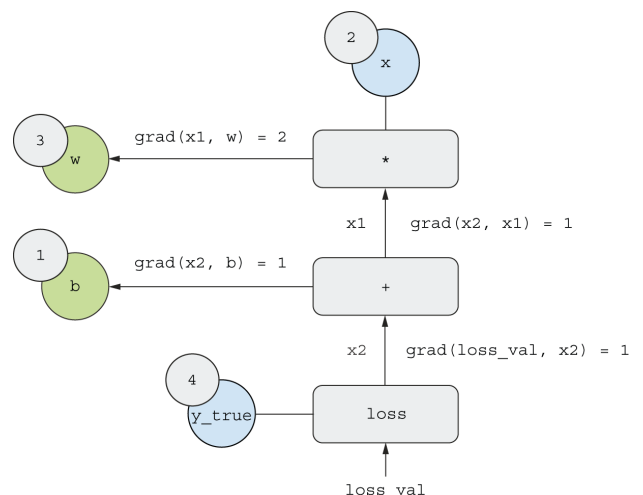
A basic example of a computation graph



Running a forward pass

Gradient Based Optimization

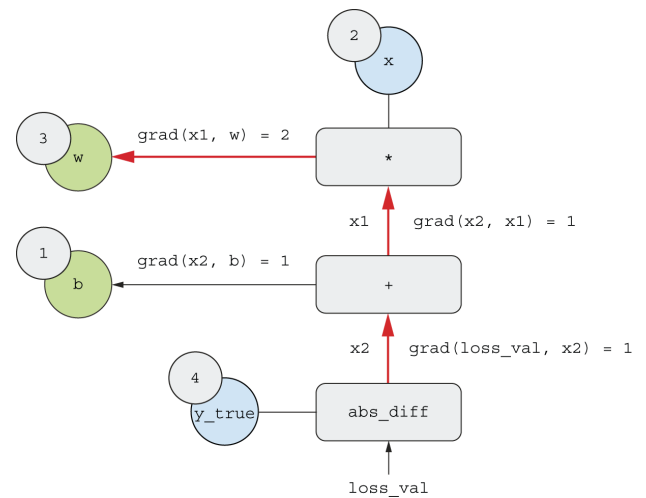
- Now let's reverse the graph
 - For each edge in the graph going from A to B , we will create an opposite edge from B to A
 - And ask, How much does B vary when A varies?
 - That is, what is $\text{grad}(B, A)$?
- $\text{grad}(\text{loss_val}, x2) = 1$
 - As $x2$ varies by an amount epsilon, $\text{loss_val} = \text{abs}(4 - x2)$ varies by the same amount.
- $\text{grad}(x1, w) = 2$
 - As w varies by an amount epsilon, $x1 = x * w = 2 * w$ varies by $2 * \text{epsilon}$.



Running a backward pass

Gradient Based Optimization

- We can obtain the derivative of a node with respect to another node by **multiplying** the **derivatives** for each edge along the path linking the two nodes.
- $\text{grad}(\text{loss_val}, w) = \text{grad}(\text{loss_val}, x2) * \text{grad}(x2, x1) * \text{grad}(x1, w)$.
- By applying the chain rule to our graph
 - $\text{grad}(\text{loss_val}, w) = 1 * 1 * 2 = 2$
 - $\text{grad}(\text{loss_val}, b) = 1 * 1 = 1$
- The name **backpropagation** comes from the fact that we **back propagate** the loss contributions of different nodes in a computation graph.



Summary

- We looked at the Hello World Example with MNIST dataset and trained and tested a simple model with Dense layers.
- We looked at Tensors and Tensor operations used in Deep Learning models.
- We looked at Gradient Based Optimization and learned about Stochastic Gradient Descent and Backpropagation Algorithm.
- We looked back at our Hello World Example and used backpropagation in Computation Graph.



References

- The content and figures used in these slides are extracted from the textbook **Deep Learning with Python, Third Edition - François Chollet.**

Next Session:
Keras, TensorFlow, PyTorch, and Jax



Any Questions?